

动态规划

《算法设计与分析》

信息科学与技术学院 曾华琳



03



矩阵连乘问题



矩阵连乘问题

问题描述:

给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$, 其中 A_i 与 A_{i+1} 是可乘的,
 $i=1, 2, \dots, n-1$ 。

如何确定矩阵连乘积的计算次序, 使得依此次序计算矩阵连乘积所需要的数乘次数最少。



矩阵连乘问题

- 矩阵乘法满足结合律
 - 矩阵连乘可以有不同的计算次序。
- 矩阵连乘的计算次序可以用加括号的方式来确定
 - 若矩阵连乘已完全加括号，则其计算次序完全确定。



矩阵连乘问题

完全加括号的矩阵连乘积可递归地定义为：

- (1) 单个矩阵是完全加括号的；
- (2) 矩阵连乘积 A 是完全加括号的，则 A 可表示为2个完全加括号的矩阵连乘积 B 和 C 的乘积并加括号，即 $A=(BC)$ 。



矩阵连乘问题

例，有四个矩阵**A**, **B**, **C**, **D**，它们的维数分别是：

$$A=50 \times 10, B=10 \times 40, C=40 \times 30, D=30 \times 5$$

连乘积**ABCD**共有五种完全加括号的方式

$(A((BC)D))$	16000	$(A(B(CD)))$	10500
--------------	--------------	--------------	--------------

$((AB)(CD))$	36000	$(((AB)C)D)$	87500
--------------	--------------	---------------	--------------

$((A(BC))D)$	34500
--------------	--------------



矩阵连乘问题

- **穷举法：**列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找到一种数乘次数最少的计算次序。
- **复杂性分析：**用 $p(n)$ 表示 n 个矩阵链乘的穷举法计算成本，如果将 n 个矩阵从第 k 和 $k+1$ 出隔开，对两个子序列再分别加扩号，则可以得到下面递归式：

$$p(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} p(k)p(n-k) & n > 1 \end{cases}$$

$\Rightarrow p(n) = C(n-1)$ 为 Catalan 数

$$C(n) = \frac{1}{n+1} \binom{2n}{n} = \Omega\left(\frac{4^n}{n^{3/2}}\right) \quad \text{呈指数增长}$$

因此，穷举法不是一个有效算法。



矩阵连乘问题

◆动态规划

将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$ ，这里 $i \leq j$ 。

考察计算 $A[i:j]$ 的最优计算次序。设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开， $i \leq k < j$ ，则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$ 。

→ **$A[i:j]$ 的计算量**： $A[i:k]$ 的计算量加上 $A[k+1:j]$ 的计算量，再加上 $A[i:k]$ 和 $A[k+1:j]$ 相乘的计算量。



矩阵连乘问题

1. 分析最优解的结构

- 特征：计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。

矩阵链乘的子问题空间： $A[i:j]$, $1 \leq i \leq j \leq n$

$A[1:1]$, $A[1:2]$, $A[1:3]$, ... , $A[1:n]$

$A[2:2]$, $A[2:3]$, ... , $A[2:n]$

...

...

...

$A[n-1:n-1]$, $A[n-1:n]$

$A[n:n]$



矩阵连乘问题

2. 建立递归关系

- 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$

取得的 k 为 $A[i:j]$ 最优次序中的断开位置, 并记录到表 $s[i][j]$ 中, 即 $s[i][j] \leftarrow k$ 。

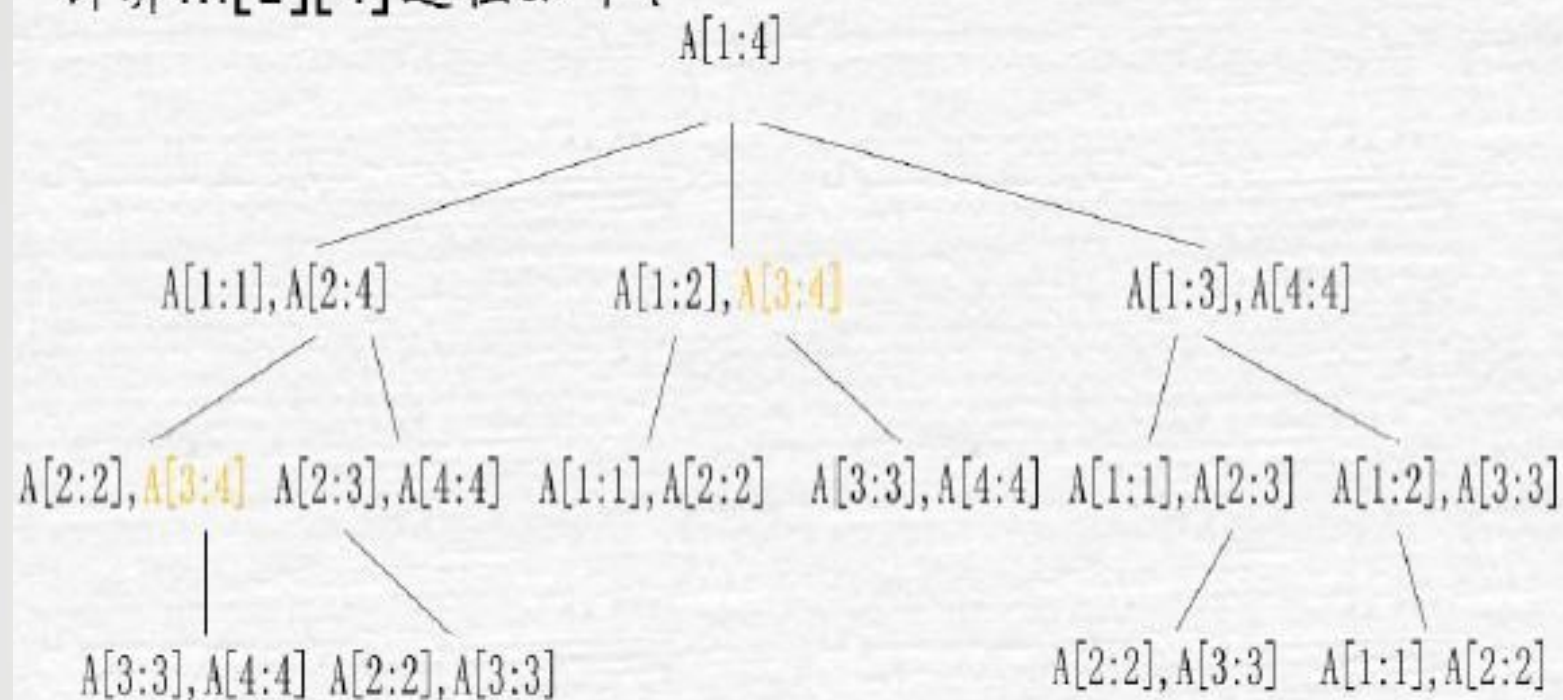
注: $m[i][j]$ 实际是子问题最优解的解值, 保存下来避免重复计算。

$$m[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i][k] + m[k+1][j] + p_{i-1}p_kp_j \} & i < j \end{cases}$$

这里,

$$(A_i A_{i+1} \dots A_k)_{p_{i-1} \times p_k} \times (A_{k+1} A_{k+2} \dots A_j)_{p_k \times p_j}$$

计算 $m[1][4]$ 过程如下:



如 $A[3:4]$ 被计算了2次, 保存下来可以节省许多时间

- 在递归计算时, 许多子问题被重复计算多次。这也是该问题可用动态规划算法求解的又一显著特征。



矩阵连乘问题

3. 计算最优值

```
void MatrixChain(int *p, int n, int **m, int **s) {  
    for (int r = 2; r <= n; r++)  
        for (int i = 1; i <= n-r+1; i++) {  
            int j=i+r-1;  
            m[i][j] = m[i+1][j]+ p[i-1]*p[i]*p[j];  
            s[i][j] = i;  
            for (int k = i+1; k < j; k++) {  
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];  
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k;}  
            }  
        }  
}
```

***p** 存放 A_i 的维数

****m** 存放最优值

****s** 存放断开位置



矩阵连乘问题

3. 计算最优值

例 计算矩阵连乘积 $A_1A_2A_3A_4A_5A_6$

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$

		j					
		1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0
(b) $m[i][j]$							
		j					
		1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0
(c) $s[i][j]$							



算法matrixChain复杂度分析:

算法的主要计算量取决于算法中对 r , i 和 k 的3重循环。循环体内的计算量为 $O(1)$, 而3重循环的总次数为 $O(n^3)$ 。

因此, 算法的计算时间上界为 $O(n^3)$ 。算法所占用的空间为 $O(n^2)$ 。



矩阵连乘问题

4. 构造最优解

按算法计算出的断点矩阵s指示的加括号方式输出计算 $A[i:j]$ 的最优先次序。

```
void Traceback(int i, int j, int **s) {  
    if (i==j) return;  
    Traceback(i, s[i][j], s);  
    Traceback(s[i][j]+1, j, s);  
    cout<<"Multiplay A"<<i<<"."<<s[i][j];  
    cout<<"and A"<<s[i][j]+1<<"."<<j<<endl;  
}
```

谢谢大家